

Correctness isn't Efficiency: Runtime Memory Divergence in LLM-Generated Code

Prateek Kumar Rajput
University of Luxembourg
Luxembourg, Luxembourg
prateek.rajpud@uni.lu

Yewei Song
University of Luxembourg
Luxembourg, Luxembourg
yewei.song@uni.lu

Abdoul Aziz
Bonkougou
University of Luxembourg
Luxembourg, Luxembourg
abdoul.bonkougou@uni.lu

Iyiola E. Olatunji
University of Luxembourg
Luxembourg, Luxembourg
emmanuel.olatunji@uni.lu

Abdoul Kader Kabore
University of Luxembourg
Luxembourg, Luxembourg
abdoulkader.kabore@uni.lu

Jacques Klein
University of Luxembourg
Luxembourg, Luxembourg
jacques.klein@uni.lu

Tegawendé F.
Bissyandé
University of Luxembourg
Luxembourg, Luxembourg
tegawende.bissyande@uni.lu

Abstract

LLMs can produce functionally correct programs, yet correctness alone does not guarantee reliability. Two programs passing the same tests can exhibit drastically different runtime behavior, creating hidden risks such as performance bottlenecks and memory leaks. Despite this, the runtime consistency of LLM-generated code remains largely unexplored. In this work, we introduce a framework to systematically quantify *execution-time memory stability* across multiple correct generations for the same task. We propose a novel solution-level metric, **DMPD (Dynamic Mean Pairwise Distance)**, which uses **Dynamic Time Warping** to compare the shapes of memory usage profiles. These profiles, which we term **Monotonic Peak Profiles (MPPs)**, are transformed to suppress transient noise, enabling robust comparison. By aggregating these scores, we derive a model-level **Model Instability Score (MIS)**. Across the BigOBench and CodeContests benchmarks, we find substantial runtime divergence among correct solutions, revealing that instability often increases with higher sampling temperatures even as pass@1 improves. We also uncover exploratory correlations between our stability metrics and established software-engineering indicators (e.g., Cognitive and Cyclo-matic Complexity), suggesting a link between operational behavior and code maintainability. These findings enable stability-aware selection of passing candidates in CI/CD pipelines, reducing operational risk without sacrificing correctness. Artifacts are available at https://github.com/pkrajpud/memory_profiling.

ACM Reference Format:

Prateek Kumar Rajput, Yewei Song, Abdoul Aziz Bonkougou, Iyiola E. Olatunji, Abdoul Kader Kabore, Jacques Klein, and Tegawendé F. Bissyandé.

* Prateek's research is in collaboration with Zortify.

† Yewei's research is in collaboration with BGL BNP Paribas.

* Aziz's research is in collaboration with B Medical Systems.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

ICSE-SEIP '26, Rio de Janeiro, Brazil

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2426-8/26/04

<https://doi.org/10.1145/3786583.3786908>

2026. Correctness isn't Efficiency: Runtime Memory Divergence in LLM-Generated Code. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE-SEIP '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3786583.3786908>

1 Introduction

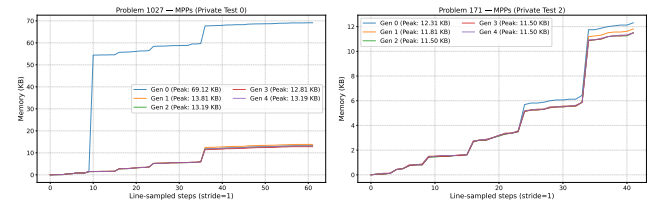


Figure 1: Monotonic Peak Profiles (MPP) on BIGOBENCH illustrating two regimes of runtime memory behavior across correct generations: left *divergence* where one MPP differs significantly (high pairwise DMPD); right *consistency* where profiles remain closely aligned (low pairwise DMPD).

Large language models (LLMs) now routinely synthesize correct programs, with evaluation dominated by execution-based metrics such as pass@k on standardized suites [8, 22, 42]. However, correctness alone does not characterize how a model *behaves* at runtime once a solution has passed tests. In production, two equally correct solutions that exhibit different memory-allocation dynamics can have materially different cost and reliability profiles. Cloud platforms often charge in proportion to configured or consumed memory, and containerized deployments routinely fail due to out-of-memory (OOM) events when limits are exceeded. Therefore, beyond functional correctness, measuring *runtime consistency* across a model's multiple valid generations for the same problem is important.

Application-level memory traces are inherently temporal and noisy. In Python, reference counting and a cyclic garbage collector trigger non-deterministic reclamation and brief oscillations even for identical logic [1, 31, 38]. System-level indicators (e.g., RSS) further blur the signal by mixing allocator policy, fragmentation, and non-Python allocations [26].

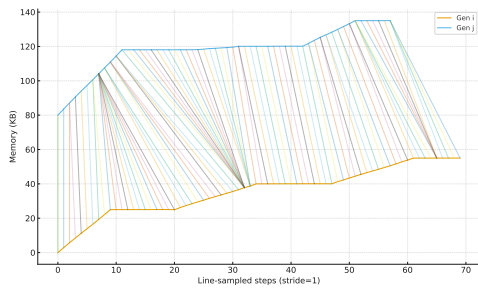


Figure 2: Example: DTW sequence alignment

To isolate contestant code while suppressing allocator churn, we instrument with `tracemalloc` and convert line-sampled current-bytes into a *Monotonic Peak Profile (MPP)*, the cumulative maximum of the baseline-corrected series (Figure 1). This transform removes downward spikes from transient frees and GC cycles, yielding a non-decreasing envelope that highlights reproducible peak-growth events across runs and environments. We then compare unit-peak MPPs by *shape*, using time-elastic Dynamic Time Warping (DTW) rather than raw magnitudes.

Why runtime stability matters. Temporal instability in execution-time memory profiles is not noise; it reflects underlying *algorithmic structure* (Refer to Figure 3 for an example case), choices of data structures, buffering, copy patterns, etc., even when solutions are functionally correct. By contrast, widely used downstream similarity metrics compare surface forms or static structure: CodeBLEU augments n-gram overlap with syntax and dataflow cues [32], while AST-based measures such as Tree Structured Edit Distance (TSED/TED) quantify structural edits between program trees [37]. These are informative for code similarity, but they do *not* assess whether multiple correct generations from the *same* model exhibit consistent behavior at runtime. Yet this is a fundamental aspect of real-world software engineering: consistency drives cost, capacity planning, tail failures, and churn-maintainability. In this work, we fill that gap by measuring runtime stability directly via shape-aware, scale-robust comparisons of memory trajectories (DMPD) and aggregating them to model-level instability (MIS). We also report how these stability proxies relate to established maintainability indicators: Cyclomatic Complexity [24], Maintainability Index [9, 28], and Cognitive Complexity [6] to ground operational variability in standard SE practice.

1.1 Industry Relevance and Scope

Much like the well-documented security vulnerabilities in generated code, which have necessitated the standardization of rigorous security scanning prior to acceptance [29, 30], operational instability constitutes a parallel risk that hinders rapid adoption. In traditional container orchestration, deployment manifests mandate precise resource specifications (requests and limits) to govern bin-packing efficiency and horizontal autoscaling [40]. Because these operational envelopes are tuned to historical baselines, an LLM-generated patch that introduces unmeasured memory variance can silently invalidate capacity estimates, forcing operators to determine if the new solution respects existing constraints or requires

expensive re-tuning. If we can design methodologies that quantify the operational risk and cost of implementing LLM-generated patches in production systems, it becomes far easier to integrate AI agents into real delivery pipelines. Our work is one such step toward that goal, offering a stability-focused signal for comparing correct patches.

We use competitive programming benchmarks, which may differ materially from industrial codebases, as they would typically avoid deep dependency graphs, long-lived architectural constraints, configuration drift, concurrency, and ecosystem-level integration testing. These differences limit direct claims about production outcomes from our work. However, they also offer a controlled lens for isolating confounding factors when analyzing stability: competitive settings allow us to compare multiple correct generations under fixed inputs, making it easier to disentangle algorithmic choice, caching behavior, allocator/GC effects, and other low-level runtime phenomena from the noise of evolving microservices and environment-specific deployment quirks. Importantly, the underlying substrate that shapes allocation dynamics, such as language runtimes, memory allocators, and garbage collection, remains the same class of mechanism that production systems depend on. From a software engineering perspective, capacity planning and safe change management benefit from metrics that capture stability beyond correctness [4, 5, 10]. We therefore position our competitive-programming results as a methodological “unit test” for instability effects that are likely to reappear in noisier industrial settings, motivating follow-up validation on large-scale industry use cases.

Our contributions are as follows.

- **Instability among correct generations.** We show that LLM-generated programs exhibit *runtime-memory* instability even when all solutions are functionally correct, and we quantify it with bounded metrics.
- **Temperature widens behavior (pass@1 vs. stability trade-off).** We show that raising sampling temperature consistently increases divergence as measured by DMPD and MIS, while often *also* improving pass@1, revealing a controllable trade-off between functional success and runtime stability.
- **Robust metrics for fair comparison.** We provide the research community with metrics (DMPD and MIS) that are robust to confounding variables such as test-case magnitude and composition. By isolating the underlying algorithmic memory structure, our metrics enable fair comparisons of program stability across diverse input scales and test suites.
- **A link to software maintainability.** We connect our operational stability proxies to established software engineering indicators.

2 Research Questions

- RQ1 Memory usage divergence.** To what extent do LLM-generated functionally correct programs generated for the same problem exhibit divergent runtime memory behaviors?
- RQ2 Impact of temperature on memory usage divergence.** How does the sampling temperature influence the consistency of runtime memory behavior across generated solutions?
- RQ3 Comparison with other metrics and baselines.** Do execution shape-aware metrics capture a novel dimension of code

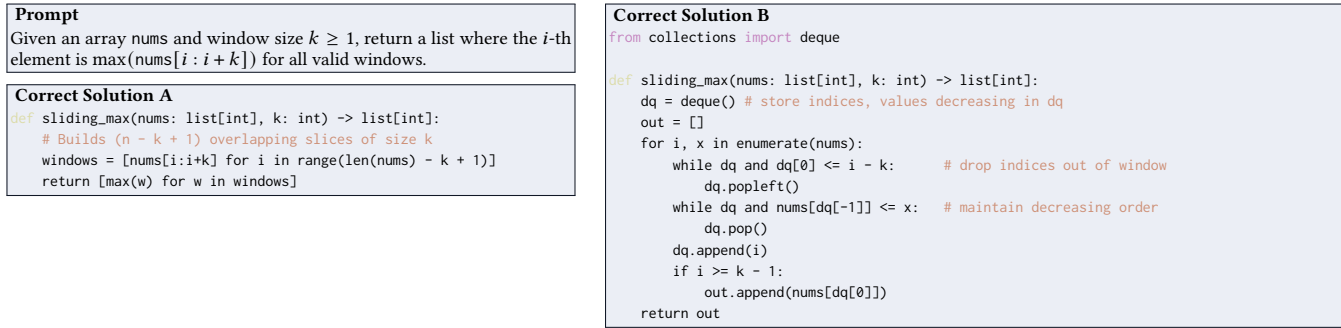


Figure 3: Example of correct solutions with runtime divergence because of different space algorithmic strategies. Solution A on the left uses $O(nk)$ space while Solution B on the right uses $O(k)$.

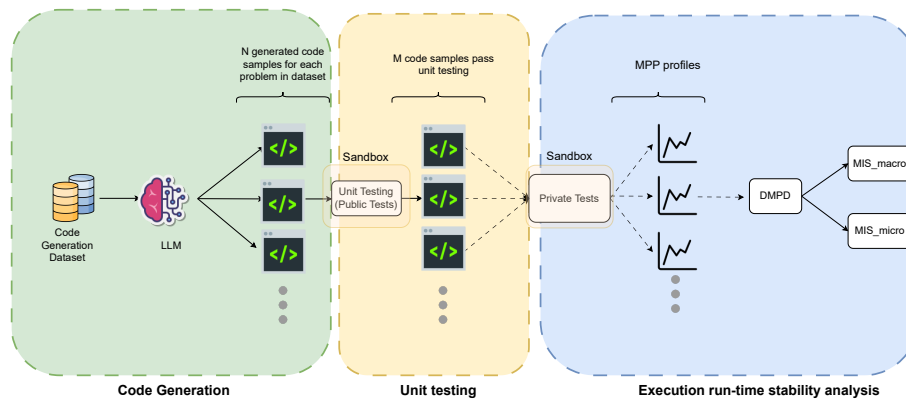


Figure 4: Pipeline for code generation and memory profiling

compared to other approaches and baselines like Normalized Peak Difference?

RQ4 Robustness to scale and instrumentation. To what degree are the instability measurements derived from our metrics influenced by variations in input scale and profiling configurations?

RQ5 Relationship between operational stability and code quality. What is the nature and strength of the association between runtime memory stability, as measured by DMPD and NMV, and established software engineering metrics of code quality?

3 Approach

Figure 4 shows the overview of our approach. We sample multiple correct solutions, run them to obtain memory-usage execution profiles (converted to monotonic peak profiles for robustness), and use Dynamic Time Warping to measure shape overlap. We define each component in detail in this section.

3.1 Dynamic Time Warping

Dynamic Time Warping (DTW) is a classic technique that aligns sequences in time [3, 19]. DTW treats two time series like flexible rubber bands laid over each other and asks: what is the *least total effort* needed to line up their shapes if we are allowed to gently stretch or compress time? At each alignment step (Refer Figure 2

for an alignment example), we pay a small *mismatch cost* for the difference in values where the two traces are currently lined up; DTW then finds the path through these steps that *minimizes* the accumulated cost. Our *Dynamic Mean Pairwise Distance* (DMPD) summarizes this by taking the *average* mismatch along DTW's best alignment between *unit-peak-normalized* MPPs, yielding a bounded score in $[0, 1]$. The unit-peak step intentionally removes scale so DMPD is driven by *shape*: where and how the profile rises, plateaus, and peaks. In practice, this separates cases with the same maximum but very different dynamics (early spike vs. late surge), yielding a robust, shape-based, and scale-robust way to measure instability. DTW's intuition "align shapes, not timestamps" is well established across domains: it originated in speech recognition to handle variable speaking rates [33], underpins classic time-series pattern matching and classification [3, 19], aligns performances to musical scores and human motion traces [25], and is routinely used to compare noisy biomedical signals such as ECGs [12, 36]. We borrow the same idea here: treat two passing solutions as two *shapes* of memory evolution, and measure how much "time-bending effort" it takes to make them agree.

3.2 Monotonic Peak Profile (MPP)

Let s_t be the traced bytes at discrete steps $t = 1, \dots, T$. After baseline correction,

$$s'_t = \max(0, s_t - s_1), \quad t = 1, \dots, T,$$

we define the **Monotonic Peak Profile (MPP)** $P = \{p_1, \dots, p_T\}$ as the cumulative maximum:

$$p_t = \begin{cases} 0 & \text{if } t = 1, \\ \max(p_{t-1}, s'_t) & \text{if } t > 1. \end{cases} \quad (1)$$

MPP is non-decreasing, emphasizes peak growth, and dampens transient alloc/free churn, improving alignment stability across runs and environments.

3.3 Pairwise Comparison via DTW on Unit-Peak Profiles

To compare the memory profiles of two programs, we first normalize their respective MPPs to isolate the shape of their memory usage from the absolute magnitude. Given two MPPs, P_a (of length n) and P_b (of length m), we scale each to have a unit peak:

$$\hat{P}_a = \frac{P_a}{\max P_a} \quad \text{and} \quad \hat{P}_b = \frac{P_b}{\max P_b}.$$

If the peak memory of a profile is zero, its normalized counterpart is defined as an all-zero sequence.

Next, we quantify the dissimilarity between these unit-peak profiles, $\hat{P}_a$ and $\hat{P}_b$, using Dynamic Time Warping (DTW). Let $D(i, j)$ be the cumulative L^1 cost for aligning the first i points of $\hat{P}_a$ with the first j points of $\hat{P}_b$. This cost is computed via the recurrence:

$$D(i, j) = |\hat{P}_{a,i} - \hat{P}_{b,j}| + \min\{D(i-1, j-1), D(i-1, j), D(i, j-1)\}, \quad (2)$$

with boundary conditions $D(0, 0) = 0$ and $D(i, 0) = D(0, j) = \infty$ for $i, j > 0$.

After computing the full cost matrix, we identify the optimal warping path, π^* , by backtracking from the final cell (n, m) . Let $|\pi^*|$ denote the length of this path, which represents the total number of aligned point-pairs between the two profiles.

Finally, we define the *pairwise Divergence in Memory Profile Dynamics (DMPD)* as the average alignment cost per step along this optimal path:

$$\text{DMPD}(P_a, P_b) = \frac{D(n, m)}{|\pi^*|}. \quad (3)$$

Since the input profiles $\hat{P}_a$ and $\hat{P}_b$ are normalized to the range $[0, 1]$, the resulting DMPD value is also bounded in $[0, 1]$, where a value of 0 indicates that the profiles have identical shapes.

3.4 Instability Aggregation

Problem-level score. For problem p , let S_p be the set of successful solutions and T_p the set of private tests. Let $\binom{S_p}{2}$ denote all *unordered* pairs of distinct solutions. For each test $t \in T_p$, we compute DMPD for every pair $(i, j) \in \binom{S_p}{2}$ using the corresponding unit-peak MPPs. The *problem-level* instability is the mean over pairs and tests:

$$D_p = \frac{1}{\left|\binom{S_p}{2}\right| \cdot |T_p|} \sum_{(i,j) \in \binom{S_p}{2}} \sum_{t \in T_p} \text{DMPD}(P_{p,i,t}, P_{p,j,t}). \quad (4)$$

Model Instability Score (MIS). Over a set of problems $\mathcal{P}$, we report the *macro* MIS as the unweighted average of problem scores:

$$\text{MIS}_{\text{macro}} = \frac{1}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} D_p. \quad (5)$$

When problems vary widely in the number of successful solutions or tests, we additionally report a *micro* average that weights by the number of evaluated pairs and tests:

$$\text{MIS}_{\text{micro}} = \frac{\sum_{p \in \mathcal{P}} \left|\binom{S_p}{2}\right| |T_p| D_p}{\sum_{p \in \mathcal{P}} \left|\binom{S_p}{2}\right| |T_p|}. \quad (6)$$

3.5 Normalized Maximum Velocity (NMV)

From the MPP, $P = \{p_t\}_{t=1}^T$, define the per-step *velocity*

$$v_t = p_{t+1} - p_t \quad (t = 1, \dots, T-1),$$

and let $\text{MaxVel}(P) = \max_t v_t$.

For fairness across solutions on the *same* private test, we normalize by that test's *median peak* (computed across solutions that produced successful runs on the test). Let $P_{p,i,\ell}$ be the MPP for problem p , solution i , and test index ℓ , and let $p_{p,i,\ell}^* = \max P_{p,i,\ell}$ denote its peak. Define the test-level median peak

$$\tilde{P}_{p,\ell} = \text{median}_{i \in S_p(\ell)} (p_{p,i,\ell}^*),$$

where $S_p(\ell)$ is the set of solutions that passed test ℓ for problem p . The **Normalized Maximum Velocity (NMV)** for (p, i, ℓ) is

$$\text{NMV}_{p,i,\ell} = \frac{\text{MaxVel}(P_{p,i,\ell})}{\tilde{P}_{p,\ell}}. \quad (7)$$

This per-test, across-solutions normalization yields comparable scales for thresholding within a test (unlike peak-normalization by $p_{p,i,\ell}^*$, (7) is not necessarily ≤ 1 ; optionally, one may clip to $[0, 1]$ if a hard bound is desired.).

Per-solution aggregation across private tests. We optionally restrict to *eligible* runs with $p_{p,i,\ell}^* \geq P_{\min}$ (e.g., $P_{\min} = 100$ KiB). Let $T'_{p,i} \subseteq T_p$ be the eligible tests for solution i on problem p . We report

$$\overline{\text{NMV}}_{p,i} = \frac{1}{|T'_{p,i}|} \sum_{\ell \in T'_{p,i}} \text{NMV}_{p,i,\ell} \quad (8)$$

where $\tau > 0$ is a configurable threshold. $\overline{\text{NMV}}_{p,i}$ summarizes the average burstiness (relative to the test's typical peak).

Beyond the formal definitions, the Algorithm below summarizes the approach in pseudocode.

Algorithm A: Code Gen $\rightarrow$ Unit Tests $\rightarrow$ DMPD Tables

Dataset: problems (problem_id, desc, public/private tests); **Settings:** models $\mathcal{M}$, temps $\mathcal{T}$, completions N , private test cap r

A. Code generation dataset (JSONL)

```

for  $m \in \mathcal{M}$ ,  $\tau \in \mathcal{T}$  do
  for problem  $P$  do
    for  $i = 1$  to  $N$  do
      prompt LLM with  $P$  at  $(m, \tau)$ ; extract Python block
      write JSON (problem_id, solution_id,  $i$ , code, tests, raw)
    end for
  end for
end for

```

B. Public unit tests (filter) run test sandbox; label each record success/fail

C. Private tests & profiling (per problem)

```

for problem  $p$  do

```



```

 $S_p \leftarrow$  solutions labeled success
for solution  $i \in S_p$  do
  for private test  $\ell \leq r$  do
    compile with filename <contestant>
    run with timeout; compare output; skip on fail/timeout/mismatch
    tracemalloc: sample current bytes restricted to <contestant>
    baseline subtract first sample; floor at 0; optional quantization  $q$ 
    transform samples  $\rightarrow$  MPP (cumulative max); record peak/max velocity
  end for
end for
Pairwise distances (per test)
for private test  $\ell$  do
  for each unordered pair  $(i, j)$  with valid MPPs do
    unit-peak normalize both MPPs
    compute DMPD via DTW with  $L^1$  local cost (avg cost along optimal path)

    append (problem  $p$ , test  $\ell$ , pair  $(i, j)$ , DMPD) to table
  end for
end for
end for
Output: per-problem, per-test pairwise DMPD tables (no aggregation)

```

Algorithm B: Aggregation to MIS_{macro} and MIS_{micro}

Inputs: **DMPD** tables from Algorithm A

A. Problem-level scores

for problem p do

gather all **DMPD** values across tests and unordered pairs

#pairs _{p} $\leftarrow$ evaluated pairs; #tests _{p} $\leftarrow$ evaluated tests

$D_p \leftarrow$ mean of **DMPD** values for p

$w_p \leftarrow$ #pairs _{p} $\times$ #tests _{p}

end for

B. Cross-problem aggregation

MIS_{macro} $\leftarrow$ mean of D_p for problems with $w_p > 0$ (equiv.: mean of all **DMPD** entries pooled across problems)

MIS_{micro} $\leftarrow$ weighted mean of D_p using weights w_p

Output: table (p, D_p, w_p) and global MIS_{macro}, MIS_{micro}

4 Experiments

Table 1: Large language models selected for our stability evaluation, grouped by source and specialization. Abbreviations are used in plots and figures.

Model	Abbreviation	Params	Context	Developer
Commercial Models				
Language/Code Models				
GPT-3.5-turbo-instruct	GPT-3.5	N/A	16k	OpenAI
GPT-4o	GPT-4o	N/A	128k	OpenAI
Claude-3.7-Sonnet	Claude-3.7-S	N/A	200k	Anthropic
Reasoning Model				
GPT-o4-mini	GPT-o4-m	N/A	128k	OpenAI
Open-Source Models				
Code Models				
Qwen2.5-Coder-7B	Qwen-7B-C	7B	64k	Alibaba Cloud
CodeLlama-7B-Instruct	CodeLlama-7B-It	7B	16k	Meta
Codestral-22B	Codestral-22B	22B	32k	Mistral AI
Language/Code Models				
Llama-3.1-8B	Llama3.1-8B	8B	128k	Meta
Mistral-7B-v0.3	Mistral-7B	7B	32k	Mistral AI
Reasoning Models				
DeepSeek-R1-Distill-Qwen-32B	DS-Qwen-32B	32B	128k	DeepSeek AI
DeepSeek-R1-Distill-Llama-70B	DS-Llama-70B	70B	128k	DeepSeek AI

Empirical Protocol. Our empirical protocol is designed to systematically quantify the *runtime memory* stability of code artifacts generated by LLMs. The fundamental task under investigation is code generation: for each experimental run, the input is a prompt containing a complete problem description drawn from our benchmarks (formal statement, I/O specification, and constraints), and the expected output is a compilable Python function intended to solve the given problem. For each problem–model pair, we generate a corpus of candidate solutions ($n = 5$ in our experiments) to analyze their collective stability. A critical prerequisite for this analysis is the establishment of *functional correctness*. We deem a

generated artifact correct if it passes all public unit tests provided by the source benchmark. Our stability study is therefore conditioned on correctness. Concretely, we profile memory-allocation dynamics for the cohort of correct solutions, transform raw traces into *Monotonic Peak Profiles* (MPP; Eq. 1), and compare unit-peak profiles with Dynamic Time Warping to obtain *DMPD* (Eq. 3), which we aggregate to per-problem scores and cross-problem MIS_{macro} and MIS_{micro} (Eqs. 4–6).

We also compute static maintainability metrics: Cyclomatic Complexity [24], Cognitive Complexity [6], and Maintainability Index [9, 28] and join them with the per-solution $\overline{\text{DMPD}}$ (mean over private tests; likewise $\overline{\text{NMV}}$). We report Pearson/Spearman correlations across all solutions (Table 4 and visualize distributional shifts with double violin plots (Figure 11) by stratifying $\overline{\text{DMPD}}/\overline{\text{NMV}}$ into tertiles (T1 low $\rightarrow$ T3 high), alongside Cliff’s δ for T1 vs. T3.

4.1 Datasets and Models

To comprehensively evaluate our approach, we utilize two distinct problem suites: CODECONTESTS and BIGOBENCH. CODECONTESTS, with its 165 problems and graded difficulty, serves as a challenging benchmark for functional correctness. In contrast, BIGOBENCH is specifically designed to probe the link between algorithmic complexity and runtime behavior; we use its 318-problem space-complexity test set. A key feature of both datasets is their inclusion of public and private unit tests. This structure allows us to first identify functionally correct solutions using the public tests and then analyze their dynamic behavior on the private ones. On these benchmarks, we evaluate a diverse set of eleven Large Language Models (LLMs) to ensure the generality of our findings (see Table 1). These models, spanning both commercial and open-source systems, are grouped into three functional categories: (1) **code-generation** models fine-tuned for synthesis, (2) general-purpose **language/code** models, and (3) **reasoning** models optimized for multi-step problem solving. This curated selection enables a robust analysis across various model scales, architectures, and specializations.

Mercury dataset. To study scale effects on MIS_{macro}, MIS_{micro}, and **DMPD**, we use the MERCURY dataset [11], whose test set contains 256 problems. MERCURY provides per-problem test-case generators, which we exploit to synthesize private tests in four strict “size” buckets $N \in \{1, 10, 100, 1000\}$. Here, N denotes the characteristic input magnitude for the primary type (e.g., list length, string length, or number of rows for list-of-lists/matrices, depending on the problem). For each problem and bucket, we generate 10 private tests. Implementation details follow a strict-size sampler: we infer an input-serialization profile from the public tests, generate candidates via the problem’s generator_code, clamp serialized inputs to the target size window, deduplicate by serialized form, and deterministically select the first K examples. Expected outputs are obtained by executing an accepted in-file reference solution.

4.2 Metric Calculation and Execution Protocol

Our experimental protocol begins with generating $N=5$ solutions for each model–problem pair across three temperatures: $\{0.0, 0.7, 0.95\}$. This selection allows us to span a *near-deterministic baseline* (0.0), a *moderate-diversity* setting widely used for code assistants (≈ 0.7 [34]), and a *high-diversity* regime (0.95) that accentuates

solution variation useful for probing the pass@1 vs. stability trade-off. To examine temperature effects more finely, we additionally perform a systematic temperature scan for gpt-3.5-turbo-instruct from 0.0 to 2.0. Each generated solution is then executed against up to $r=10$ private tests, using a single-bin input-magnitude policy with a fixed seed to ensure each run occupies a comparable scale regime. During execution, we instrument application-level memory via `tracemalloc` (*current bytes*) and scope attribution to the contestant module. Execution is guarded by per-run timeouts; runs that fail (timeout, OOM, mismatch, or instrumentation error) are excluded from aggregation but retained in logs. The collected memory traces are then transformed into MPPs (Eq. 1) after baseline-subtraction and quantization. From these MPPs, we compute pairwise distances as *DMPD* (Eq. 3) using DTW with an L^1 local cost. Finally, these scores are aggregated to form the per-problem instability metric D_p (Eq. 4) and the cross-problem aggregates MIS_{macro} and MIS_{micro} (Eqs. 5–6).

Practical note on measurement noise. Even with application-level scoping, memory traces reflect interpreter and allocator effects (reference counting, GC cycles), library behavior, scheduling, and incidental system activity. Our MPP transform and unit-peak, time-elastic alignment substantially reduce, but cannot eliminate this variability. Accordingly, our defaults prioritize stability: *line sampling* with stride $s=1$, byte quantization $q=64$ B, strict filename scoping, and unconstrained DTW with L^1 costs; we report distributional summaries (medians/IQRs) and sensitivity ablations (including time sampling with $\Delta t \in \{0.1, 1, 2\}$ ms and coarser q) to separate model-induced divergence from residual profiling noise.

4.3 Profiling Hyperparameters

We convert raw memory traces into stable profiles using two hyperparameters that control quantization and sampling. To reduce allocator noise, we **quantize** memory by rounding each sample to the nearest multiple of q bytes (default $q=64$ B); larger q yields smoother profiles but lower resolution. To align memory with program structure rather than wall-clock effects, we sample memory every s -th executed line in the contestant’s module (**line stride**, default $s=1$), which reduces sensitivity to machine speed. We compared this to time-based sampling and found line-based sampling more stable. Unless stated otherwise, we report results with $s=1$ and $q=64$ B.

5 Results

5.1 Memory Usage Divergence

Restricting analysis to *passing* solutions does not collapse behavior to a single runtime mode: for the same problem, we observe cohorts whose MPP envelopes either cluster tightly or diverge markedly (Figure 1). This dispersion is captured by non-zero pairwise DMPD values and, when aggregated, by MIS, which summarizes how much the shapes of memory growth differ among correct generations (Figures 5 and 6) for the complete dataset.

Crucially, *functional* success and *operational* stability decouple. Models with similar pass@1 can exhibit very different instability magnitudes. For example, on BIGOBENCH, GPT-4o increases pass@1 from 0.68 ($T=0$) to 0.80 ($T=0.95$), while its MIS_{macro} simultaneously rises from 0.0042 to 0.0072 (Table 2), indicating more

varied memory-evolution shapes at higher temperature. By contrast, Claude-3.7-Sonnet attains higher pass@1 with a lower MIS, showing that two models with comparable correctness can differ materially in runtime stability.

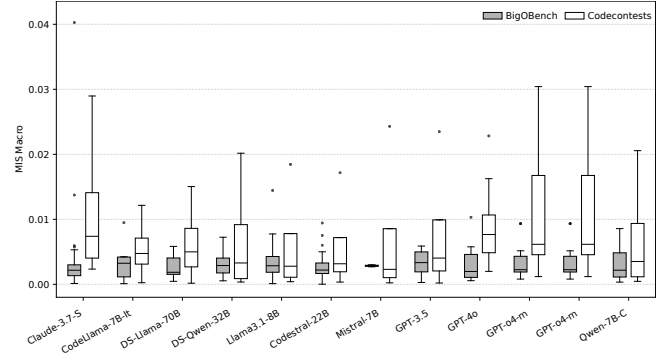


Figure 5: MIS macro for BigOBench and CodeContests

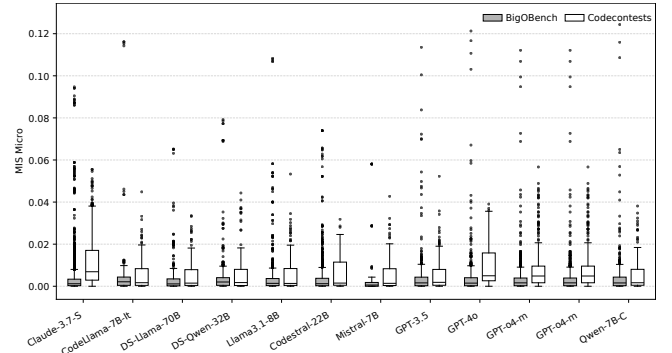


Figure 6: MIS micro for BigOBench and CodeContests

Implication. In production, two correct programs can carry different memory costs. Instability among correct generations therefore impacts capacity planning; correctness thus is a necessary gate, not a sufficient guarantee of *operability*. Stability-aware selection (e.g., reranking by MIS/DMPD) can reduce unseen real-world software engineering risks without sacrificing pass@1 (effect size on maintainability is discussed in Section 5.5).

Answer to RQ1

Among functionally correct solutions, we observe substantial divergence in memory trajectories. Pairwise DMPD is non-zero and aggregate MIS varies by model and temperature (Figures 1, 5 and 6, Table 2). Correctness does not imply runtime stability.

5.2 Impact of Temperature on Memory Usage Divergence

Sampling temperature T —the standard knob for exploring an LLM’s internal solution space—is ubiquitous in practice; we therefore study how T shapes runtime *stability* among correct generations. We consistently observe that an increase in temperature widens runtime behavior among correct generations. Across models and both suites,

MIS_{macro} and MIS_{micro} generally increase with temperature (Figure 7). For instance, on BigOBench, GPT-o4-mini shows MIS_{macro} growing from 0.0029 ($T=0$) to 0.0126 ($T=0.95$) while $pass@1$ also rises (0.69→0.80), indicating temperature perturbs execution *paths* (data structures, control flow), not just surface syntax. For gpt-3.5-turbo-instruct, we scan $T \in [0, 2]$; beyond $T \approx 1.4$ it does not yield ≥ 2 correct generations for any problem, precluding DMPD/MIS computation (which require at least two passing solutions).

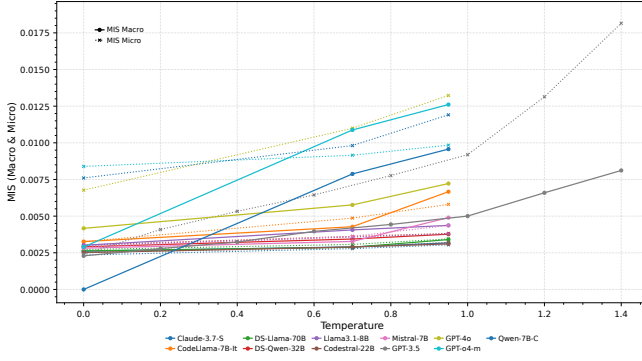


Figure 7: MIS macro and micro relation with temperature on BigOBench

Implication. Higher temperature explores a broader solution space: $pass@1$ may improve, but the same diversity amplifies runtime-memory instability (higher DMPD).

Answer to RQ2

Higher temperature increases instability. Both MIS_{macro} and MIS_{micro} rise with sampling temperature across most model–dataset pairs (Figure 7, Table 2), reflecting broader solution-manifold exploration that diversifies memory dynamics.

5.3 Comparison With Other Metrics and Baselines

MIS is largely orthogonal to surface/syntactic similarity (n-gram/Code BLEU [32], and structure similarity (AST similarity, TSED [37])). See correlation heatmap (Figure 8), indicating that it captures a different, execution-time dimension of code behavior.

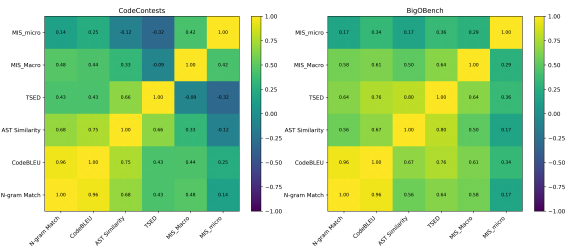


Figure 8: Pearson correlation heatmap for GPT-o4-mini on traditional metrics vs MIS on BigOBench

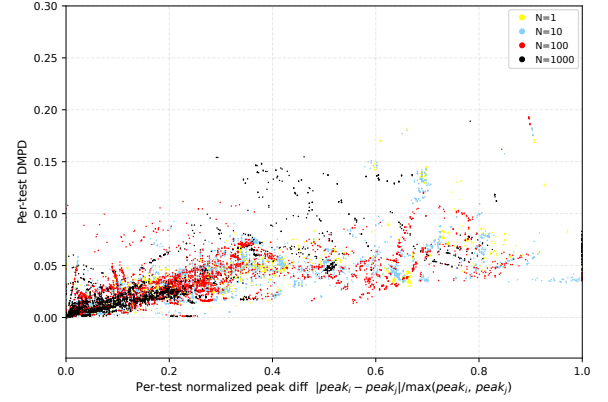


Figure 9: Openai-o4-mini DMPD vs Normalized peak difference with scale (on Mercury dataset)

Peak-only proxies such as normalized peak difference (NPD) vary widely with the test case and input scale. In Figure 9, NPD shows large, test-dependent spread, whereas DMPD clusters tightly, reflecting its *scale invariance*. Although we observe a strong but imperfect correlation ($R^2 \approx 0.75$) between per-problem mean NPD and DMPD i.e., solutions unstable in shape often also fluctuate in amplitude, the amplitude proxy is far more sensitive to the particular private tests, while DMPD is not. Consistently, across explicit scale factors $N \in \{1, 10, 100, 1000\}$, Figure 10 shows MIS remains fairly stable with scale, mirroring DMPD’s behavior (with additional damping from aggregation).

Implication. Shape-based, time-elastic metrics (DMPD/MIS) are more *robust* indicators of runtime instability across test suites and input scales than peak-based proxies.

Answer to RQ3

Orthogonal to traditional metrics and more robust than baseline. DMPD/MIS capture orthogonal dimension missed by traditional metrics (Figure 8), and remain stable under input scaling (Figures 9 and 10), whereas peak-memory based baseline varies widely with specific tests.

5.4 Robustness to Scale and Instrumentation

Ablations show that MIS is fairly stable with scaled input sizes of test suites (Figure 10), but profiling choices can inflate or dampen perceived instability (Table 3). Relative to line sampling with stride $s=1$ and 64 B quantization (baseline), increasing stride to $s=5-10$ modestly raises MIS (+5–9%) and inflates dispersion. Coarsening byte quantization to 128–256 B causes negligible ($<1\%$) effect. In contrast, fine-grained *time* sampling at 0.1 ms overstates instability (+46% macro / +55% micro), consistent with allocator/GC micro-effects [31]; 1–2 ms reduces but does not eliminate the bias.

Implication. Instability estimates should reflect *model/code* diversity, not measurement noise. Our defaults (line sampling, $s=1$, modest quantization) provide conservative, reproducible estimates;

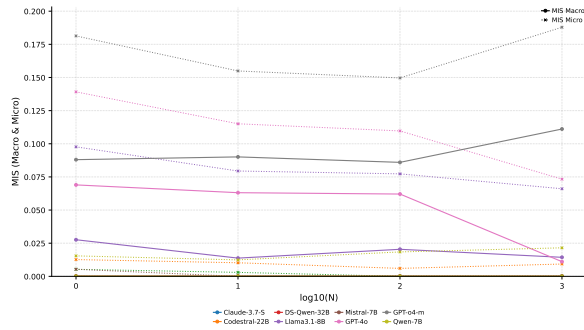


Figure 10: MIS micro and macro with scale (on Mercury dataset)

time sampling is appropriate only when wall-clock constraints dominate and intervals are sufficiently coarse.

Answer to RQ4

Conclusions are robust under reasonable settings, but measurement matters. Prefer line sampling and modest quantization; avoid very fine time grids that amplify allocator noise (Table 3).

5.5 Relationship Between Operational Stability and Code Quality

As discussed before higher (DMPD) values indicate *greater instability*, now higher (NMV) values can be thought of as *greater burstiness*. We form tertiles on the *raw* proxy values separately for DMPD and NMV (T1=lowest, T3=highest). Hence, “moving from T1 to T3” means *decreasing* stability (for DMPD) or *increasing* burstiness (for NMV). Cliff’s δ is reported as $\delta(T1, T3)$; thus $\delta < 0$ means the SE metric is *larger at higher instability/burstiness*. The double-violin plot in Figure 11 shows the spread and relationship for Claude.

Across models, we observe two robust regularities under NMV tertiles: (i) *Cognitive* and *Cyclomatic* complexities consistently increase as burstiness rises ($\delta < 0$ in all reported models; $\rho > 0$), and (ii) *Maintainability Index* (MI) also tends to increase with burstiness (negative δ with small-to-moderate positive ρ) (Refer to Table 4). Under DMPD tertiles, the most stable pattern concerns MI: higher DMPD is associated with *higher* MI, whereas the relations to complexity metrics are comparatively weak and model-dependent (signs of δ are mixed and magnitudes are smaller). Taken together, these results indicate that the two proxies emphasize different facets of code behavior: NMV (burstiness) aligns strongly with structural complexity, while DMPD aligns more with MI.

Answer to RQ5

Instability is moderately linked to SE quality. Higher NMV (burstiness) tracks *higher* Cognitive/Cyclomatic complexity, while higher DMPD (higher instability) aligns mainly with *higher* MI; links to complexity under DMPD are weak/inconsistent.

6 Discussion

6.1 Cliffs of Correctness

LLM-authored code often exhibits *correctness cliffs*: under a fixed prompt, tiny perturbations like temperature or random seed can flip a sample from fail to pass, yielding sharp jumps in $\text{pass}@k$ as k increases [8]. The same stochastic variability extends beyond correctness to *operational shape*. Two candidates that both pass unit tests may realize markedly different allocation patterns (temporary buffers, cache lifetimes, data structure choices). At small N these differences look benign, yet modest input or traffic shifts can push one variant over a tail threshold: an OOM kill, paging storm, or latency blowup, while the other remains stable. Classic systems guidance reminds us that user and business impact are dominated by *tails*, not means [10].

Use in post-training. Ideally, the manifold of *correct* solutions should be *flat*, meaning nearby samples behave similarly at runtime rather than riddled with cliffs. Although injecting execution signals directly into pretraining is costly and often non-differentiable, they can be exploited in post-training (e.g., policy gradient methods like PPO [35]) to smooth execution effects by rewarding low DMPD (down-weighting unstable candidates) in post-training. We do not claim to cover all operational risks. Instead, we report associations between runtime-stability proxies (DMPD/MIS) and established SE maintainability metrics, and leave causal links to production SLOs as future work.

6.2 Implications for Auto-Generated Software

Our results show that instability affects *software qualities* even when functional tests pass. As burstiness rises (NMV: T1→T3), we consistently observe *higher* Cognitive and Cyclomatic complexity, while Maintainability Index (MI) also trends upward. Additionally, lower stability by DMPD is most reliably associated with *higher* MI, with weak links to cognitive or cyclomatic complexity.

Why this relation matters in practice. In a typical pipeline, teams gate on "working" solutions, shipping whichever sample happens to pass the tests. Yet two passing candidates can embody very different *runtime stories*: one grows memory smoothly; another idles for a while and then surges late. Nothing breaks at small inputs, so the difference might be invisible until a real-life load situation nudges the spikier variant over a limit. The result is not just an isolated OOM or retry cascade; it is a stream of hidden costs that accumulate over time—extra headroom in capacity planning, flaky builds, noisy alerts, and engineering toil. These effects compound across services and releases, creating operational drag that correctness metrics simply do not surface [4, 10, 27].

Practical selection. Because these costs add up, it is worth choosing not just a *working* candidate, but the *most stable* one and, where possible, the model that is most stable for the task at hand. A lightweight policy is enough: for each patch, sample k correct generations, run a brief single-bin scale check, compute DMPD/NMV, and select the candidate with low median and tight spread (subject to correctness and basic latency checks). At the model level, prefer those with lower MIS on your workload. This adds minimal overhead to CI but directly reduces the risk that small, everyday changes snowball into production incidents [5, 14].

Table 2: Comparative Analysis of LLMs: pass@1, MIS_macro, and MIS_micro across BigOBench and CodeContests

Model	pass@1						MIS_macro						MIS_micro					
	BigO			CC			BigO			CC			BigO			CC		
	0	0.7	0.95	0	0.7	0.95	0	0.7	0.95	0	0.7	0.95	0	0.7	0.95	0	0.7	0.95
Commercial Models																		
<i>Language/Code Models</i>																		
GPT-4o	0.68	0.73	0.80	0.15	0.17	0.19	0.0042	0.0058	0.0072	0.0097	0.0091	0.0130	0.0068	0.0110	0.0132	0.0097	0.0091	0.0130
Claude-3.7-Sonnet	0.77	0.81	0.87	0.22	0.23	0.25	0.0026	0.0029	0.0032	0.0113	0.0107	0.0100	0.0023	0.0028	0.0030	0.0113	0.0107	0.0100
gpt-3.5-turbo-instruct	0.35	0.38	0.43	-	-	-	0.0023	-	-	0.0010	0.0043	0.0050	0.0024	-	-	0.0020	0.0072	0.0089
<i>Reasoning Model</i>																		
GPT-o4-mini	0.69	0.72	0.80	0.11	0.11	0.12	0.0029	0.0109	0.0126	0.0096	0.0113	0.0115	0.0084	0.0092	0.0098	0.0096	0.0113	0.0115
Open Source Models																		
<i>Code Models</i>																		
Qwen2.5-Coder-7B	0.39	0.41	0.46	0.05	0.05	0.06	0.0000	0.0079	0.0096	0.0050	0.0004	0.0005	0.0076	0.0098	0.0119	0.0050	0.0004	0.0005
CodeLlama-7B-Instruct	0.10	0.11	0.12	0.01	0.01	0.01	0.0033	0.0043	0.0067	0.0095	0.0067	0.0139	0.0033	0.0049	0.0058	0.0095	0.0067	0.0139
Codestral-22B	0.48	0.50	0.55	0.09	0.10	0.11	0.0025	0.0029	0.0031	0.0072	0.0060	0.0072	0.0025	0.0029	0.0032	0.0072	0.0060	0.0072
<i>Language/Code Models</i>																		
Llama-3.1-8B	0.28	0.30	0.34	0.06	0.06	0.07	0.0030	0.0041	0.0044	0.0015	0.0020	0.0022	0.0030	0.0036	0.0038	0.0015	0.0020	0.0022
Mistral-7B-v0.3	0.10	0.11	0.12	0.01	0.01	0.01	0.0028	0.0033	0.0049	0.0080	0.0052	0.0102	0.0029	0.0036	0.0044	0.0080	0.0052	0.0102
<i>Reasoning Models</i>																		
DeepSeek-R1-Distill-Llama-70B	0.13	0.14	0.15	0.01	0.01	0.01	0.0026	0.0029	0.0034	0.0075	0.0045	0.0071	0.0027	0.0031	0.0034	0.0075	0.0045	0.0071
DeepSeek-R1-Distill-Qwen-32B	0.15	0.15	0.16	0.01	0.01	0.01	0.0029	0.0034	0.0038	0.0083	0.0053	0.0079	0.0030	0.0036	0.0038	0.0083	0.0053	0.0079

Table 3: Sensitivity of instability metrics to sampling knobs (BigOBench) for GPT-o4-mini at temp = 0.7

Setting	Mode	Stride s	Quant. q (B)	Interval Δt (ms)	MIS_macro	Δ macro %	MIS_micro	Δ micro %	Paired $\tilde{\Delta}D_p$ [IQR]	p / δ
Baseline	Line	1	64	-	0.0045	-	0.0065	-	-	-
Stride $\uparrow$	Line	5	64	-	0.0047	+5%	0.0070	+8%	+0.00015 [0.0018]	$p=0.06 / \delta=0.16$
	Line	10	64	-	0.0048	+6%	0.0071	+9%	+0.00018 [0.0018]	$p=0.05 / \delta=0.18$
Quantization $\uparrow$	Line	1	128	-	0.0045	+0.5%	0.0066	+0.8%	+0.00002 [0.0002]	$p=0.70 / \delta=0.02$
	Line	1	256	-	0.0045	+0.8%	0.0066	+1.0%	+0.00003 [0.0002]	$p=0.62 / \delta=0.03$
Time sampling (matched density)	Time	-	64	0.1	0.0066	+46%	0.0101	+55%	+0.00160 [0.0018]	$p<0.001 / \delta=0.62$
	Time	-	64	1	0.0053	+18%	0.0080	+23%	+0.00090 [0.0012]	$p=0.010 / \delta=0.36$
	Time	-	64	2	0.0050	+10%	0.0074	+14%	+0.00055 [0.0008]	$p=0.07 / \delta=0.22$

Notes: Paired $\tilde{\Delta}D_p$ = median across problems of $D_p^{\text{setting}} - D_p^{\text{baseline}}$; IQR = interquartile range of these differences; p = Wilcoxon signed-rank p -value; δ = Cliff's delta.

Table 4: Stability proxies vs. SE metrics

Model	Cognitive Complexity (mean)		Maintainability Index		Cyclomatic Complexity (mean)	
	DMPD ($\rho \delta$)	NMV ($\rho \delta$)	DMPD ($\rho \delta$)	NMV ($\rho \delta$)	DMPD ($\rho \delta$)	NMV ($\rho \delta$)
Claude-3.7-Sonnet	-0.094 0.121	0.334 -0.405	0.180 -0.260	0.050 -0.125	-0.134 0.187	0.255 -0.288
Codestral-22B	0.098 -0.055	0.413 -0.497	0.187 -0.296	0.035 -0.165	0.071 -0.016	0.346 -0.401
DeepSeek-R1-Distill-Llama-70B	0.316 -0.377	0.545 -0.671	0.752 -0.889	0.362 -0.502	0.024 -0.066	0.455 -0.571
DeepSeek-R1-Distill-Qwen-32B	0.433 -0.388	0.767 -1.000	0.128 -0.265	0.232 -0.297	0.310 -0.250	0.697 -0.918
Llama-3.1-8B	-0.178 0.110	0.471 -0.550	0.354 -0.461	0.528 -0.707	-0.288 0.304	0.429 -0.465

Reading guide: Tertiles are computed *per model* and *separately* for DMPD (instability) and NMV (burstiness): T1=low proxy (more stable), T3=high proxy (less stable). Cells report Spearman's ρ and Cliff's δ (T1, T3); since T3 is less stable, $\delta < 0$ means the SE metric tends to be *higher* when instability/burstiness is higher.

6.3 Why DMPD robustness matters in industry.

DMPD offers a version-aware signal that is comparatively less confounded by the *scale* and *composition* of the test workload than peak or normalized peak memory. Indeed, in large repositories, peak memory can fluctuate primarily because the workload grows, the test suite evolves, or integration coverage shifts. This makes peak-based comparisons fragile for pre vs post patch assessment. By emphasizing the *shape* change of memory-profile behavior between versions, DMPD provides a more test condition-agnostic view of whether a patch meaningfully alters execution dynamics, which is

precisely the sort of comparative evidence that teams seek when reasoning about operational risk before release.

7 Threats to Validity

Language and runtime specificity. All experiments use Python and instrument application-level allocations via `tracemalloc` under CPython's reference counting plus cyclic GC [31, 38]. These choices suppress RSS/allocator noise but also bias us toward Python's memory semantics. Other ecosystems differ materially e.g., JVM and .NET employ moving, generational collectors with tunable heuristics [16, 18], Go prioritizes low-latency concurrent

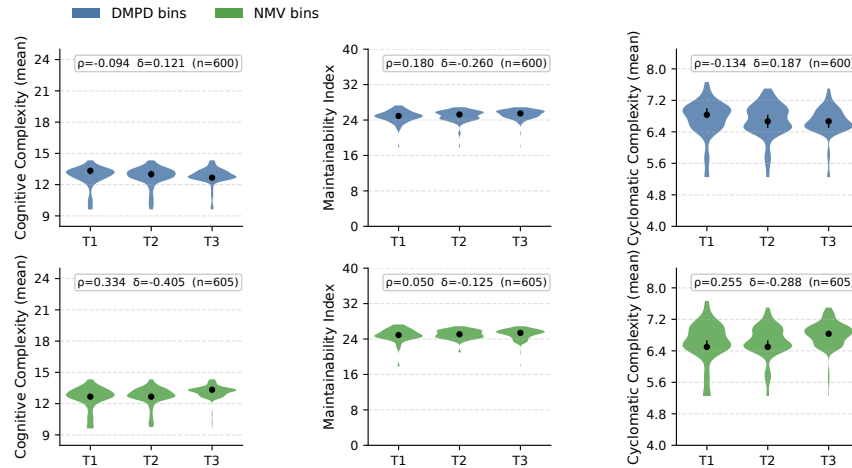


Figure 11: Claude-3.7-Sonnet – Stability tertiles (T1 low → T3 high) vs SE metrics

GC [13], while Rust enforces ownership and largely avoids GC [21]. Such differences can change allocation shapes and, therefore, DTW distances. Low-level allocator behavior and fragmentation also vary by platform [18, 26]. Hence, our scores cannot be transferred to other languages without replication.

Benchmark vs. real systems. We evaluate on simple competitive coding style problems (CodeContests, BigOBench), not on production services. Real software is more complex and introduces frameworks, concurrency, and deployment constraints (containers, limits, autoscaling) that may interact with memory trajectories and tail behavior [10]. External validity is a general challenge in SE empiricism [20, 41]. Our findings motivate, but do not replace, validation on industrial repositories and end-to-end workloads.

Sampling budget and statistical power. We cap generations to each problem at $N=5$ correct generations and at most $r=10$ private tests. This modest budget limits $\binom{k}{2}$ pairwise comparisons, can miss long-tail behaviors, and increases the variance of D_p and MIS. Although we fix seeds and use a single-bin policy to improve repeatability (for Mercury scaling test), estimates remain sensitive to which samples happen to be drawn. We report medians/IQRs and perform sensitivity checks, but larger N/r or explicit uncertainty estimates would yield tighter measurements.

8 Related Work

Research on code-generating LLMs has moved from “does it work?” to “how well does it behave?”, with a growing emphasis on properties beyond pure I/O correctness. We outline this trajectory below.

Phase I: Functional correctness at scale. Early benchmarks established that LLMs can produce *passing* programs, measuring $\text{pass}@k$ on HumanEval and MBPP [2, 8]. Large-scale competitive settings (e.g., AlphaCode/CodeContests) operationalized this lens by filtering massive candidate pools through public/private tests [22]. These efforts treat all passing solutions as equivalent and do not distinguish *how* they behave at runtime.

Phase II: Text and structure beyond I/O. To differentiate among multiple passing outputs, evaluation broadened to surface form and program structure. CodeBLEU augments n-gram matching with

syntax. [32]. Structural similarity metrics: AST-based measures and tree-edit distances such as TSED quantify how two programs differ in static structure [17, 37]. Empirical audits further showed that LLM code, while correct, can be brittle or suboptimal [39]. These approaches, however, remain *representation-level* proxies; they do not look at dynamic behavior under execution.

Phase III: Execution-grounded signals. Recent work exploits runtime signals directly. AlphaCode leverages test outcomes to triage candidates [22]; AlphaDev optimizes measured latency to discover faster algorithms [23]; energy-aware generation explores efficiency objectives [15]. In parallel, prompting for algorithmic complexity (BIGOBENCH) connects predicted complexity to time/space behavior [7]. Outside the LLM literature, mature profilers (e.g., Python tracemalloc/PEP 454 and Valgrind/Massif) provide the instrumentation needed to observe allocation dynamics [26, 38]. Despite these ingredients, very few systematically quantify the stability of runtime memory behavior across multiple correct generations of the same specification.

9 Conclusion

Correctness isn’t consistency: even *passing* LLM programs can diverge in runtime memory behavior. We introduced MPP (to denoise traces) and a shape-based DMPD aggregated as MIS (with NMV for burstiness) to quantify this instability. Beyond being a measurement gap, instability imposes *hidden software-engineering costs* extra capacity headroom, flaky builds, noisy alerts, incident toil, and long-run maintainability drag—that $\text{pass}@k$ and text/syntax metrics do not surface. Practically, teams should pick the *stable* passing candidate (low DMPD/NMV) and prefer models with lower MIS, e.g., via stability-aware reranking and conservative temperatures. Looking forward, extending these ideas beyond Python/memory to latency/CPU/I/O and linking stability to SLOs and incident data will better translate runtime shape into operational risk.

References

- [1] 2025. tracemalloc — Trace memory allocations. <https://docs.python.org/3/library/tracemalloc.html>.
- [2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le,

- et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).
- [3] Donald J Berndt and James Clifford. 1994. Using dynamic time warping to find patterns in time series. In *Proceedings of the 3rd international conference on knowledge discovery and data mining*. 359–370.
 - [4] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. 2016. *Site reliability engineering: how Google runs production systems*. " O'Reilly Media, Inc."
 - [5] Betsy Beyer, Niall Richard Murphy, David K Rensin, Kent Kawahara, and Stephen Thorne. 2018. *The site reliability workbook: practical ways to implement SRE*. " O'Reilly Media, Inc."
 - [6] G Ann Campbell. 2018. Cognitive complexity: An overview and evaluation. In *Proceedings of the 2018 international conference on technical debt*. 57–58.
 - [7] Pierre Chambon, Baptiste Roziere, Benoit Sagot, and Gabriel Synnaeve. 2025. BigO (Bench)—Can LLMs Generate Code with Controlled Time and Space Complexity? *arXiv preprint arXiv:2503.15242* (2025).
 - [8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
 - [9] Don Coleman, Dan Ash, Bruce Lowther, and Paul Oman. 1994. Using metrics to evaluate software system maintainability. *Computer* 27, 8 (1994), 44–49.
 - [10] Jeffrey Dean and Luiz André Barroso. 2013. The tail at scale. *Commun. ACM* 56, 2 (2013), 74–80.
 - [11] Mingzhe Du, Anh Tuan Luu, Bin Ji, Qian Liu, and See-Kiong Ng. 2024. Mercury: A code efficiency benchmark for code large language models. *Advances in Neural Information Processing Systems* 37 (2024), 16601–16622.
 - [12] Toni Giorgino. 2009. Computing and visualizing dynamic time warping alignments in R: the dtw package. *Journal of Statistical Software* 31 (2009), 1–24.
 - [13] Richard Hudson. 2015. Go GC: Prioritizing low latency and simplicity. *The Go Programming Language Blog*. Retrieved 21 (2015).
 - [14] Jez Humble and David Farley. 2010. *Continuous delivery: reliable software releases through build, test, and deployment automation*. Pearson Education.
 - [15] Shashikant Ilager, Lukas Florian Briem, and Ivona Brandic. 2025. Green-Code: Learning to Optimize Energy Efficiency in Llm-Based Code Generation. In *2025 IEEE 25th International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, 559–569.
 - [16] Sanath Jayasena, Milinda Fernando, Tharindu Rusira, Chalitha Perera, and Chamara Philips. 2015. Auto-tuning the java virtual machine. In *2015 IEEE International Parallel and Distributed Processing Symposium Workshop*. IEEE, 1261–1270.
 - [17] Lingxiao Jiang, Ghassan Misherghi, Zhendong Su, and Stephane Glondy. 2007. Deckard: Scalable and accurate tree-based detection of code clones. In *29th International Conference on Software Engineering (ICSE'07)*. IEEE, 96–105.
 - [18] Richard Jones, Antony Hosking, and Eliot Moss. 2023. *The garbage collection handbook: the art of automatic memory management*. Chapman and Hall/CRC.
 - [19] Eamonn Keogh and Chotirat Ann Ratanamahatana. 2005. Exact indexing of dynamic time warping. *Knowledge and information systems* 7, 3 (2005), 358–386.
 - [20] Barbara Kitchenham, Stuart Charters, et al. 2007. Guidelines for performing systematic literature reviews in software engineering. (2007).
 - [21] Steve Klabnik and Carol Nichols. 2023. *The Rust programming language*. No Starch Press.
 - [22] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.
 - [23] Daniel J Mankowitz, Andrea Michi, Anton Zhernov, Marco Gelmi, Marco Selvi, Cosmin Paduraru, Edouard Leurent, Shariq Iqbal, Jean-Baptiste Lespiau, Alex Ahern, et al. 2023. Faster sorting algorithms discovered using deep reinforcement learning. *Nature* 618, 7964 (2023), 257–263.
 - [24] Thomas J McCabe. 1976. A complexity measure. *IEEE Transactions on software Engineering* 4 (1976), 308–320.
 - [25] Meinard Müller. 2007. *Information retrieval for music and motion*. Springer.
 - [26] Nicholas Nethercote and Julian Seward. 2007. Valgrind: A Framework for Heavy-weight Dynamic Binary Instrumentation. In *PLDI*.
 - [27] Michael Nygard. 2018. Release it!: design and deploy production-ready software. (2018).
 - [28] Paul Oman and Jack Hagemester. 1992. Metrics for assessing a software system's maintainability. In *Proceedings Conference on Software Maintenance 1992*. IEEE Computer Society, 337–338.
 - [29] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2025. Asleep at the keyboard? assessing the security of github copilot's code contributions. *Commun. ACM* 68, 2 (2025), 96–105.
 - [30] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do users write more insecure code with ai assistants?. In *Proceedings of the 2023 ACM SIGSAC conference on computer and communications security*. 2785–2799.
 - [31] Python Software Foundation. 2025. gc — Garbage Collector interface. <https://docs.python.org/3/library/gc.html>. Accessed: 2025-09-26.
 - [32] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297* (2020).
 - [33] Hiroaki Sakoe and Seibi Chiba. 2003. Dynamic programming algorithm optimization for spoken word recognition. *IEEE transactions on acoustics, speech, and signal processing* 26, 1 (2003), 43–49.
 - [34] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems* 36 (2023), 68539–68551.
 - [35] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
 - [36] Pavel Senin. 2008. Dynamic time warping algorithm review. *Information and Computer Science Department University of Hawaii at Manoa Honolulu, USA* 855, 1-23 (2008), 40.
 - [37] Yewei Song, Cedric Lothritz, Daniel Tang, Tegawendé F Bissyandé, and Jacques Klein. 2024. Revisiting code similarity evaluation with abstract syntax tree edit distance. *arXiv preprint arXiv:2404.08817* (2024).
 - [38] Victor Stinner. 2014. PEP 454: Add a new tracemalloc module to trace Python memory allocations. <https://peps.python.org/pep-0454/>.
 - [39] Haoye Tian, Weiqi Lu, Tsz On Li, Xunzhu Tang, Shing-Chi Cheung, Jacques Klein, and Tegawendé F Bissyandé. 2023. Is ChatGPT the ultimate programming assistant—how far is it? *arXiv preprint arXiv:2304.11938* (2023).
 - [40] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. 2015. Large-scale cluster management at Google with Borg. In *Proceedings of the tenth european conference on computer systems*. 1–17.
 - [41] Claes Wohlin and Per Runeson. 2012. M. Hest, MC Ohlsson, B. Regnell, and A. Wesslén, Experimentation in software engineering.
 - [42] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. 2024. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877* (2024).